

アノテーションツール インストールガイド

CAVT INSTALLATION

QIN HUANGYI

内容

簡単なインストールガイド	2
Ubuntu 22.04/20.04 (x86_64/amd64)	2
Windows 10	4
Mac OS Mojave	6
高度なトピック	8
CVAT ソースコードの取得方法	8
CVAT ヘルスチェックコマンド	9
プロキシの背後で CVAT をデプロイする	10
Traefik ダッシュボードの使用	10
追加コンポーネント	11
すべてのコンテナを停止する	11
独自のドメインを使用する	11
共有パス	11
メール検証	12
Scaleway パブリッククラウドに CVAT をデプロイする	13
HTTPS を使用した安全な CVAT インスタンスのデプロイ	13
外部データベースを使用した CVAT のデプロイ	13
CVAT イメージのプル/ビルド/更新方法	14
トラブルシューティング	15
中国ユーザー向けの情報源	15
証明書が原因で HTTPS が機能しない	17

簡単なインストールガイド

CVAT を使用する前に、インストールする必要があります。以下のドキュメントには、最も一般的なオペレーティングシステム向けの手順が含まれています。お使いのシステムがこのドキュメントでカバーされていない場合でも、以下の手順を他のシステムに適用するのは比較的簡単はずです。

プロキシサーバーを経由している場合は、以下の手順を修正する必要があるかもしれません。プロキシは高度なトピックであり、このガイドでは扱っていません。

中国からのアクセスについては、[中国ユーザー向けの情報源](#)セクションをお読みください。

Ubuntu 22.04/20.04 (x86_64/amd64)

- 端末ウィンドウを開きます。Ubuntu で端末ウィンドウを開く方法がわからない場合は、[こちらの回答](#)をお読みください。
- 以下のコマンドを端末ウィンドウに入力して、Docker と Docker Compose をインストールします。詳細な手順は[こちら](#)で確認できます。

```
sudo apt-get update
sudo apt-get --no-install-recommends install -y \
  apt-transport-https \
  ca-certificates \
  curl \
  gnupg-agent \
  software-properties-common
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key
add -
sudo add-apt-repository \
  "deb [arch=amd64] https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) \
  stable"
sudo apt-get update
sudo apt-get --no-install-recommends install -y \
  docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

- (オプション) Docker コマンドの前に `sudo` を付けるのを避けるには、[インストール後の手順](#)を実行できます。これには、`docker` という名前の Unix グループを作成し、現在のユーザーをこのグループに追加することが含まれます。

```
sudo groupadd docker
sudo usermod -aG docker $USER
```

ログアウトして再度ログイン（または再起動）し、グループメンバーシップが再評価されるようにします。その後、端末ウィンドウで `groups` コマンドを入力し、出力に `docker` グループが含まれているか確認できます。

- Git を使用して [GitHub リポジトリ](#)から `_CVAT_` のソースコードをクローンします。

以下のコマンドは最新の `develop` ブランチをクローンします：

```
git clone https://github.com/cvat-ai/cvat
cd cvat
```

リリースバージョンの一つをダウンロードしたい場合や、`wget` または `curl` ツールを使用したい場合は、[代替手段](#)を参照してください。

- ネットワーク経由または異なるシステムから CVAT にアクセスするには、`CVAT_HOST` 環境変数をエクスポートします。

```
export CVAT_HOST=FQDN またはあなたの IP アドレス
```

- Docker コンテナを実行します。最新の CVAT およびその他の必要なイメージ（`postgres`、`redis` など）をダウンロードし、コンテナを開始するには時間がかかります。

```
docker compose up -d
```

- (オプション) `CVAT_VERSION` 環境変数を使用して、インストールしたい CVAT の特定のバージョン（例：`v2.1.0`、`dev`）を指定します。デフォルトの動作：`develop` ブランチには `dev` イメージが、リリースバージョンには対応するリリースイメージがプルされます。

```
CVAT_VERSION=dev docker compose up -d
```

- 代替：未リリースの変更を含むイメージをローカルでビルドしたい場合は、[CVAT イメージのプル/ビルド/更新方法](#)セクションを参照してください。
- ユーザーを登録できますが、デフォルトではタスクのリストを表示する権限がありません。したがって、スーパーユーザーを作成する必要があります。スーパーユーザーは管理パネルを使用して、ユーザーに適切なグループを割り当てることができます。以下のコマンドを使用してください：

```
docker exec -it cvat_server bash -ic 'python3 ~/manage.py createsuperuser'
```

管理者アカウントのユーザー名とパスワードを選択してください。詳細については、[Django ドキュメント](#)をお読みください。

- Google Chrome は CVAT がサポートする唯一のブラウザです。これもインストールする必要があります。以下のコマンドを端末ウィンドウに入力してください：

```
curl https://dl-ssl.google.com/linux/linux_signing_key.pub | sudo apt-key add -
sudo sh -c 'echo "deb [arch=amd64] http://dl.google.com/linux/chrome/deb/ stable main" >> /etc/apt/sources.list.d/google-chrome.list'
sudo apt-get update
sudo apt-get --no-install-recommends install -y google-chrome-stable
```

- インストールされた Google Chrome ブラウザを開き、[localhost:8080](#) にアクセスします。ログインページでスーパーユーザーのログイン情報/パスワードを入力し、_Login_ボタンを押します。これで新しいアノテーションタスクを作成できるようになります。詳細については、[{{< ilink "/docs/manual" "CVAT マニュアル">}}](#)をお読みください。

Windows 10

- [こちらの公式ガイド](#)を参照して WSL2（Linux 用 Windows サブシステム）をインストールします。WSL2 には Windows 10、バージョン 2004 以降が必要です。WSL2 をインストールした後、選択した Linux ディストリビューションをインストールします。
- [Docker Desktop for Windows](#) をダウンロードしてインストールします。Docker for Windows Installer をダブルクリックしてインストーラーを実行します。詳

細な手順は[こちら](#)で確認できます。Docker WSL2 バックエンドの公式ガイドは[こちら](#)で確認できます。注意：Docker には具体的に WSL2 バックエンドを使用していることを確認してください。

- Docker Desktop で、Settings >> Resources >> WSL Integration に移動し、選択した Linux ディストリビューションとの統合を有効にします。
- [Git for Windows](#) をダウンロードしてインストールします。パッケージをインストールする際は、すべてのオプションをデフォルトのままにしてください。パッケージに関する詳細情報は[こちら](#)で確認できます。
- [Google Chrome](#) をダウンロードしてインストールします。これは CVAT がサポートする唯一のブラウザです。
- Windows メニューに移動し、インストールした Linux ディストリビューションを見つけて実行します。端末ウィンドウが表示されるはずです。
- [GitHub リポジトリ](#) から `_CVAT_` のソースコードをクローンします。

以下のコマンドは最新の develop ブランチをクローンします：

```
git clone https://github.com/cvat-ai/cvat
cd cvat
```

リリースバージョンの一つをダウンロードしたい場合は、[代替手段](#)を参照してください。

- Docker コンテナを実行します。最新の CVAT リリースおよびその他の必要なイメージ (postgres、redis など) を DockerHub からダウンロードし、コンテナを作成するには時間がかかります。

```
docker compose up -d
```

- (オプション) CVAT_VERSION 環境変数を使用して、インストールしたい CVAT の特定のバージョン (例：v2.1.0、dev) を指定します。デフォルトの動作：develop ブランチには dev イメージが、リリースバージョンには対応するリリースイメージがプルされます。

```
CVAT_VERSION=dev docker compose up -d
```

- 代替：未リリースの変更を含むイメージをローカルでビルドしたい場合は、[CVAT イメージのプル/ビルド/更新方法](#)セクションを参照してください。
- ユーザーを登録できますが、デフォルトではタスクのリストを表示する権限がありません。したがって、スーパーユーザーを作成する必要があります。スーパーユーザーは管理パネルを使用して、他のユーザーに適切なグループを割り当てることができます。以下のコマンドを使用してください：

```
sudo docker exec -it cvat_server bash -ic 'python3 ~/manage.py createsuperuser'
```

`winpty` がインストールされていない場合や上記のコマンドが機能しない場合は、以下を試すこともできます：

```
# 最初に docker イメージに入る
docker exec -it cvat_server /bin/bash
# 次に実行
python3 ~/manage.py createsuperuser
```

管理者アカウントのユーザー名とパスワードを選択してください。詳細については、[Django ドキュメント](#)をお読みください。

- インストールされた Google Chrome ブラウザを開き、[localhost:8080](#) にアクセスします。ログインページでスーパーユーザーのログイン情報/パスワードを入力し、`_Login_` ボタンを押します。これで新しいアノテーションタスクを作成できるようになります。詳細については、["/docs/manual" "CVAT マニュアル">](#)をお読みください。

Mac OS Mojave

- [Docker for Mac](#) をダウンロードします。`Docker.dmg` をダブルクリックしてインストーラーを開き、Moby the whale（クジラのアイコン）を Applications フォルダにドラッグします。Applications フォルダ内の `Docker.app` をダブルクリックして Docker を起動します。詳細な手順は[こちら](#)で確認できます。
- Mac に Git をインストールする方法はいくつかあります。最も簡単なのは、おそらく Xcode Command Line Tools をインストールすることです。Mavericks (10.9)

以降では、初めてターミナルから `git` を実行しようとする、これを簡単に行うことができます。

```
git --version
```

まだインストールされていない場合は、インストールを促すプロンプトが表示されます。詳細な手順は[こちら](#)で確認できます。

- [Google Chrome](#) をダウンロードしてインストールします。これは **CVAT** がサポートする唯一のブラウザです。
- 端末ウィンドウを開きます。ターミナルアプリは **Applications** フォルダ内の **Utilities** フォルダにあります。開くには、**Applications** フォルダを開き、**Utilities** を開いてターミナルをダブルクリックするか、**Command - スペースバー**を押して **Spotlight** を起動し、「**Terminal**」と入力して検索結果をダブルクリックします。
- **Git** を使用して [GitHub](#) リポジトリから `_CVAT_` のソースコードをクローンします。

以下のコマンドは最新の `develop` ブランチをクローンします：

```
git clone https://github.com/cvat-ai/cvat
cd cvat
```

リリースバージョンの一つをダウンロードしたい場合や、`wget` または `curl` ツールを使用したい場合は、[代替手段](#)を参照してください。

- **Docker** コンテナを実行します。最新の **CVAT** リリースおよびその他の必要なイメージ (`postgres`、`redis` など) を **DockerHub** からダウンロードし、コンテナを作成するには時間がかかります。

```
docker compose up -d
```

- (オプション) `CVAT_VERSION` 環境変数を使用して、インストールしたい **CVAT** の特定のバージョン (例：`v2.1.0`、`dev`) を指定します。デフォルトの動作：`develop` ブランチには `dev` イメージが、リリースバージョンには対応するリリースイメージがプルされます。

```
CVAT_VERSION=dev docker compose up -d
```

- 代替：未リリースの変更を含むイメージをローカルでビルドしたい場合は、[CVAT イメージのプル/ビルド/更新方法](#)セクションを参照してください。
- ユーザーを登録できますが、デフォルトではタスクのリストを表示する権限すらありません。したがって、スーパーユーザーを作成する必要があります。スーパーユーザーは管理パネルを使用して、他のユーザーに適切なグループを割り当てることができます。以下のコマンドを使用してください：

```
docker exec -it cvat_server bash -ic 'python3 ~/manage.py createsuperuser'
```

管理者アカウントのユーザー名とパスワードを選択してください。詳細については、[Django ドキュメント](#)をお読みください。

- インストールされた Google Chrome ブラウザを開き、[localhost:8080](#) にアクセスします。ログインページでスーパーユーザーのログイン情報/パスワードを入力し、[_Login_](#) ボタンを押します。これで新しいアノテーションタスクを作成できるようになります。詳細については、[CVAT マニュアル](#)をお読みください。

高度なトピック

CVAT ソースコードの取得方法

Git (Linux, Mac, Windows)

1. システムに Git がまだインストールされていない場合はインストールします。
 - Ubuntu:
`sudo apt-get --no-install-recommends install -y git`
 - Windows: <https://git-scm.com/download/win> の手順に従います。
2. [GitHub リポジトリ](#) から `_CVAT_` のソースコードをクローンします。

以下のコマンドはデフォルトブランチ（`develop`）をクローンします：

```
git clone https://github.com/cvat-ai/cvat
cd cvat
```

特定のタグ、例えば `v2.1.0` をクローンするには：

```
git clone -b v2.1.0 https://github.com/cvat-ai/cvat
cd cvat
```

Wget (Linux, Mac)

最新の develop ブランチをダウンロードするには：

```
wget https://github.com/cvat-ai/cvat/archive/refs/heads/develop.zip
unzip develop.zip && mv cvat-develop cvat
cd cvat
```

特定のタグをダウンロードするには：

```
wget https://github.com/cvat-ai/cvat/archive/refs/tags/v1.7.0.zip
unzip v1.7.0.zip && mv cvat-1.7.0 cvat
cd cvat
```

Curl (Linux, Mac)

最新の develop ブランチをダウンロードするには：

```
curl -LO https://github.com/cvat-ai/cvat/archive/refs/heads/develop.zip
unzip develop.zip && mv cvat-develop cvat
cd cvat
```

特定のタグをダウンロードするには：

```
curl -LO https://github.com/cvat-ai/cvat/archive/refs/tags/v1.7.0.zip
unzip v1.7.0.zip && mv cvat-1.7.0 cvat
cd cvat
```

CVAT ヘルスチェックコマンド

以下のコマンドを使用すると、CVAT コンテナが正常に動作しているかテストできます。

```
docker exec -t cvat_server python manage.py health_check
```

正常な CVAT コンテナの期待される出力：

```
Cache backend: default    ... working
DatabaseBackend           ... working
DiskUsage                 ... working
MemoryUsage              ... working
MigrationsHealthCheck    ... working
OPAHealthCheck            ... working
```

プロキシの背後で CVAT をデプロイする

プロキシの背後で CVAT をデプロイし、自動アノテーションのためのサーバーレス関数を使用しない場合は、エクスポートされた環境変数 `http_proxy`、`https_proxy`、`no_proxy` でイメージをビルドするのに十分です。そうでない場合は、コンテナを起動するユーザーのホームディレクトリにある `~/.docker/config.json` ファイルを作成または編集し、次のような JSON を追加してください：

```
{
  "proxies": {
    "default": {
      "httpProxy": "http://proxy_server:port",
      "httpsProxy": "http://proxy_server:port",
      "noProxy": "/*.test.example.com,.example2.com"
    }
  }
}
```

これらの環境変数は、任意のコンテナ内で自動的に設定されます。詳細については、[Docker ドキュメント](#)を参照してください。

Traefik ダッシュボードの使用

`docker-compose` ファイルをカスタマイズしていて予期しない問題が発生した場合、Traefik ダッシュボードを使用すると、問題が Traefik の設定にあるのか、それとも他のサービスにあるのかを確認するのに非常に役立ちます。

`docker-compose.yml` から以下の行のコメントを解除することで、Traefik ダッシュボードを有効にできます。

```
services:
  traefik:
    # Traefik ダッシュボードを取得するにはコメントを解除
    # - "--entryPoints.dashboard.address=:8090"
    # - "--api.dashboard=true"
    # labels:
    # - traefik.enable=true
    # - traefik.http.routers.dashboard.entrypoints=dashboard
    # - traefik.http.routers.dashboard.service=api@internal
    # - traefik.http.routers.dashboard.rule=Host(`${CVAT_HOST:-localhost}`)
```

そして、`docker-compose.https.yml` を使用している場合は、これらの行もコメント解除します。

```
services:
  traefik:
    command:
      # Traefik ダッシュボードを取得するにはコメントを解除
      # - "--entryPoints.dashboard.address=:8090"
      # - "--api.dashboard=true"
```

注意：この「安全でない」ダッシュボードは、本番環境（およびインスタンスが公開されている場合）では推奨されません。本番環境でダッシュボードを保持したい場合は、Traefik のドキュメントを読んで適切に保護する方法を確認してください。

追加コンポーネント

半自動および自動アノテーション

{{< ilink “/docs/administration/advanced/installation_automatic_annotation” “このガイド” >}}に従ってください。

すべてのコンテナを停止する

以下のコマンドは、`up` によって作成されたコンテナとネットワークを停止し、削除します。

```
docker compose down
```

独自のドメインを使用する

`localhost` 以外（別のドメイン）から CVAT インスタンスにアクセスしたい場合は、`CVAT_HOST` 環境変数を次のように指定する必要があります：

```
export CVAT_HOST=<YOUR_DOMAIN>
```

共有パス

タスクを作成する際に、データのアップロードに共有ストレージを使用できます。そのためには、共有ストレージを CVAT docker コンテナにマウントする必要があります。この目的のための `docker-compose.override.yml` の例：

```

services:
  cvat_server:
    volumes:
      - cvat_share:/home/django/share:ro
  cvat_worker_import:
    volumes:
      - cvat_share:/home/django/share:ro
  cvat_worker_export:
    volumes:
      - cvat_share:/home/django/share:ro
  cvat_worker_annotation:
    volumes:
      - cvat_share:/home/django/share:ro
  cvat_worker_chunks:
    volumes:
      - cvat_share:/home/django/share:ro

volumes:
  cvat_share:
    driver_opts:
      type: none
      device: /mnt/share
      o: bind

```

共有デバイスパスを実際の共有パスに変更できます。

クラウドストレージを FUSE として{{< ilink

“/docs/administration/advanced/mounting_cloud_storages” “マウント”>}}し、後で共有として使用できます。

メール検証

新しく登録されたユーザーに対してメール検証を有効にすることができます。Django allauth を設定してメール検証を有効にするには、[設定ファイル](#)でこれらのオプションを指定します（ACCOUNT_EMAIL_VERIFICATION = ‘mandatory’）。ユーザーのメールアドレスが検証されるまでアクセスは拒否されます。

```

ACCOUNT_AUTHENTICATION_METHOD = 'username_email'
ACCOUNT_CONFIRM_EMAIL_ON_GET = True
ACCOUNT_EMAIL_REQUIRED = True
ACCOUNT_EMAIL_VERIFICATION = 'mandatory'

```

Django 用のメールバックエンド設定

```

EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'

```

また、メールを送信するために Django のメールバックエンドを設定する必要があります。これは使用しているメールサーバーに依存し、このチュートリアルではカバーされていません。詳細については、[Django SMTP バックエンド設定](#)を参照してください。

Scaleway パブリッククラウドに CVAT をデプロイする

このチュートリアルに従って、Scaleway クラウドインスタンスに CVAT をインストールし、マウントされたオブジェクトストレージバケット内のデータを使用してリモートアクセスを設定します。

HTTPS を使用した安全な CVAT インスタンスのデプロイ

Traefik を使用すると、Let's Encrypt からドメインの TLS 証明書を自動的に取得でき、ウェブサイトへのアクセスに HTTPS プロトコルを使用できるようになります。

これを有効にするには、まず CVAT_HOST（ウェブサイトのドメイン）と ACME_EMAIL（Let's Encrypt の連絡先メールアドレス）環境変数を設定します：

```
export CVAT_HOST=<YOUR_DOMAIN>
export ACME_EMAIL=<YOUR_EMAIL>
```

次に、docker-compose.https.yml ファイルを使用して、ベースの docker-compose.yml ファイルをオーバーライドします：

```
docker compose -f docker-compose.yml -f docker-compose.https.yml up -d
```

ファイアウォールでは、ポート 80 と 443 が任意の場所からのインバウンド接続に対して開いている必要があります。

すると、CVAT インスタンスはあなたのドメインのポート 443（HTTPS）と 80（HTTP、443 にリダイレクト）で利用可能になります。

外部データベースを使用した CVAT のデプロイ

デフォルトでは、docker compose up は PostgreSQL データベースサーバーを起動し、CVAT のデータを保存するために使用されます。代わりに独自の PostgreSQL インスタンスを使用したい場合は、次のようにします。CVAT は docker-compose.yml で使用されている PostgreSQL と同じメジャーバージョンのみをサポートすることに注意してください。

まず、データベース接続設定を持つ環境変数を定義します：

```
export CVAT_POSTGRES_HOST=<PostgreSQL ホスト名> # 必須
export CVAT_POSTGRES_PORT=<PostgreSQL ポート> # デフォルトは 5432
export CVAT_POSTGRES_DBNAME=<PostgreSQL データベース名> # デフォルトは "cvat"
export CVAT_POSTGRES_USER=<PostgreSQL ロール名> # デフォルトは "root"
export CVAT_POSTGRES_PASSWORD=<PostgreSQL ロールのパスワード> # 必須
```

次に、docker-compose.external_db.yml ファイルを docker compose up コマンドに追加します：

```
docker compose -f docker-compose.yml -f docker-compose.external_db.yml up -d
```

CVAT イメージのプル/ビルド/更新方法

- **CVAT バージョンが 2.1.0 以下の場合**、compose 設定は常に最新のイメージタグを指しているため、docker を使用してイメージをプルする必要があります。

例：

```
docker pull cvat/server:v1.7.0
docker tag cvat/server:v1.7.0 openvino/cvat_server:latest
```

```
docker pull cvat/ui:v1.7.0
docker tag cvat/ui:v1.7.0 openvino/cvat_ui:latest
```

CVAT バージョンが v2.1.0 より大きい場合、CVAT_VERSION 環境変数を使用して、DockerHub から特定のバージョンのビルド済みイメージをプルすることができます（例：dev）：

```
CVAT_VERSION=dev docker compose pull
```

- イメージを自分でビルドするには、docker-compose.dev.yml compose 設定ファイルを docker compose コマンドに含めます。これは、ソースコードに変更を加えた CVAT をビルドする場合に便利です。

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml build
```

- ローカルイメージを latest または dev タグに更新するには、以下を実行します：

```
CVAT_VERSION=dev docker compose pull
```

または

```
CVAT_VERSION=latest docker compose pull
```

トラブルシューティング

中国ユーザー向けの情報源

中国にお住まいの場合、インストールには以下の情報源を上書きする必要があります。

- `apt update` を使用するために：

[Ubuntu ミラーリングヘルプ](#)

コンパイル済みパッケージ：

```
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal main restricted
universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal-updates main res
tricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal-backports main r
estricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal-security main re
stricted universe multiverse
```

またはソースパッケージ：

```
deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal main restric
ted universe multiverse
deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal-updates main
restricted universe multiverse
deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal-backports ma
in restricted universe multiverse
deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal-security mai
n restricted universe multiverse
```

- [Docker ミラーステーション](#)

`daemon.json` ファイルにレジストリミラーを追加します：

```
{
  "registry-mirrors": [
    "http://f1361db2.m.daocloud.io",
    "https://docker.mirrors.ustc.edu.cn",
    "https://hub-mirror.c.163.com",
    "https://mirror.ccs.tencentyun.com"
```

```
]
}
```

- pip を使用するために：

[PyPI ミラーリングヘルプ](#)

```
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

- npm を使用するために：

[npm ミラーリングヘルプ](#)

```
npm config set registry https://registry.npm.taobao.org/
```

- git の代わりに gitee を使用：

[gitee.com 上の CVAT リポジトリ](#)

- 高速化ソース [docker.com](#) を置き換えるために実行：

```
curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | sudo
apt-key add -
sudo add-apt-repository \
    "deb [arch=amd64] https://mirrors.aliyun.com/docker-ce/linux/ubuntu \
    $(lsb_release -cs) stable"
```

(注: [download.docker.com](#) を一般的な中国のミラーである [aliyun](#) に置き換えました。必要に応じて調整してください。)

- 高速化ソース [google.com](#) を置き換えるために実行：(注: [google.com](#) GPG キーの直接的な代替は、ローカルネットワーク設定によっては難しい場合があります。多くの場合、中国のミラーで設定されたパッケージマネージャーはこれを暗黙的に処理します。[dl-ssl.google.com](#) への直接アクセスがブロックされている場合、ユーザーはキーを取得する別の方法を見つけるか、キーをキャッシュしているミラーを利用する必要があるかもしれません。元のコマンドは参考のために残していますが、中国のユーザーは状況に応じて対応する必要があります。)

```
curl https://dl-ssl.google.com/linux/linux_signing_key.pub | sudo apt-key
add -
```

証明書が原因で HTTPS が機能しない

SSL 接続に問題がある場合、原因を特定するには、次のコマンドを実行して traefik からログを取得する必要があります：

```
docker logs traefik
```

ログは問題を見つけるのに役立ちます。

エラーがファイアウォールに関連している場合：

- ポート 80 と 443 を任意の場所からのインバウンド接続に対して開きます。
- acme.json を削除します。場所はおそらく次のようになります：
/var/lib/docker/volumes/cvat_cvat_letsencrypt/_data/acme.json。

acme.json が削除された後、すべての cvat docker コンテナを停止します：

```
docker compose -f docker-compose.yml -f docker-compose.https.yml down
```

変数が（あなたの値で）設定されていることを確認します：

```
export CVAT_HOST=<YOUR_DOMAIN>  
export ACME_EMAIL=<YOUR_EMAIL>
```

そして docker を再起動します：

```
docker compose -f docker-compose.yml -f docker-compose.https.yml up -d
```